

0073A- Compilation Techniques

Lecturer: **Andrea Corradini**

andrea.corradini@unipi.it

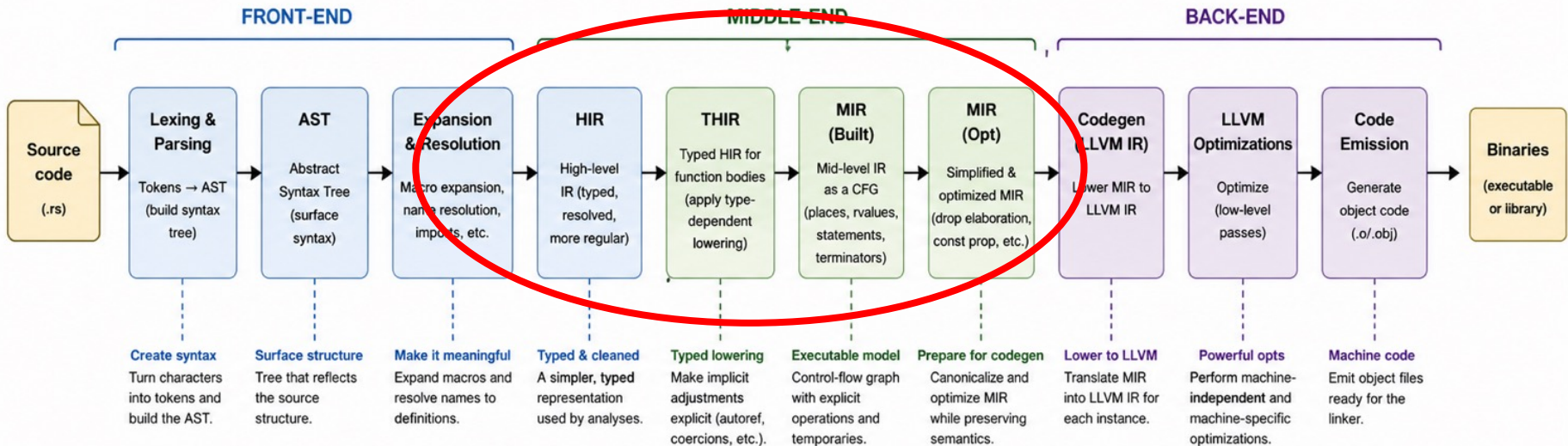
<http://pages.di.unipi.it/corradini/>

CompTech-26: RUST #6

Aspects of Rust Compilation (5)

- Linting and errors
- Type System of Rust
- Type Checking in `rustc`
- Type Inference in `rustc`
- Some words about Hindley-Milner and Algorithm W
- Other things:
 - RustBelt
 - Poloniu
 - Bugs in Rust and `rustc`
- *Type inference in uHaskell*

The rustc Compilation Pipeline



● FRONT-END - Understand the program text. - Resolve names and types. - Produce HIR for the crate.	● MIDDLE-END - Build and transform MIR. - Run borrow checking and other analyses. - Prepare MIR for codegen.	● BACK-END - Monomorphize generic code. - Generate LLVM IR. - Optimize and emit object code.	● AFTER rustc - Link object files and libraries. - Produce final binaries.	Key idea Ownership and borrowing are enforced statically in the middle-end over MIR, well before code reaches the backend.
--	--	--	---	---

AST = Abstract Syntax Tree • HIR = High-Level IR • THIR = Typed HIR • MIR = Mid-Level IR • CFG = Control-Flow Graph

Linting and Error Diagnostics in Rust

In software, a “lint” is a tool used to help improve your source code.

- `rustc` contains a number of lints: during compilation it also runs the lints.
- These lints may produce a warning, an error, or nothing at all, depending on the configuration.

```
$ cat main.rs
fn main() {
    let x = 5;
}
$ rustc main.rs
warning: unused variable: `x`
--> main.rs:2:9
   |
2  |     let x = 5;
   |         ^
   |
= note: `#[warn(unused_variables)]` on by default
= note: to avoid this warning, consider using `_x` instead
```

Lints versus fixed diagnostics

- Some messages are emitted via [lints](#), where the user can control the level. Most diagnostics are hard-coded such that the user cannot control the level.
- Usually it is obvious whether a diagnostic should be “fixed” or a lint, but there are some grey areas.

Examples:

- [Borrow checker errors](#): these are fixed errors. The user cannot adjust the level of these diagnostics to silence the borrow checker.
- [Dead code](#): this is a lint. While the user probably doesn’t want dead code in their crate, making this a hard error would make refactoring and development very painful.
- [future-incompatible lints](#): these are silenceable lints. Warnings are emitted, and will eventually be turned into fixed (hard) errors.

Lint Levels

In rustc, lints are divided into six *levels*:

1. **allow** → do nothing
2. **expect** → see later
3. **warn** → produces a warning, compilation proceeds (eg: unused variable)
4. **force-warn** → like **warn**, but level cannot be changed
5. **deny** → produces an error
6. **forbid** → like **deny**, but level cannot be changed

Each lint has a **default level**, and the compiler has a default **warning level**.

expect level

Allows to check that a specific lint is emitted. Eg, in debugging, or before deleting a lint.

- Suppresses the expected lint if emitted
- Emits a warning if it is not emitted

```
fn main() {  
    #[expect(unused_variables)]    // outer attribute  
                                   // `unused_variables` lint emitted:  
    let unused = "Everyone ignores me";    // prints nothing  
  
    #[expect(unused_variables)] // `unused_variables` lint is not emitted:  
    let used = "I'm useful";    // expectation not fulfilled -> warning  
    println!("The `used` value is equal to: {:?}", used);  
}
```

warning: this lint expectation is unfulfilled

--> src/main.rs:7:14

```
|  
7 |     #[expect(unused_variables)]  
|           ^^^^^^^^^^^^^^^^^^^  
|
```

= note: `#[warn(unfulfilled\_lint\_expectations)]` on by default

Configuring warning levels

- Lints can be configured to behave as for a level different from the defined one
- Via **compiler flags**
 - The `-A`, `-W`, `--force-warn` `-D`, and `-F` flags let you turn one or more lints into *allowed*, *warning*, *force-warn*, *deny*, or *forbid* levels.

```
$ rustc lib.rs --crate-type=lib -D missing-docs // usually allow
error: missing documentation for crate
--> lib.rs:1:1
  |
1 | pub fn foo() {}
  | ^^^^^^^^^^^^^^
  |
  = note: requested on the command line with `-D missing-docs`

error: missing documentation for a function
--> lib.rs:1:1
  |
1 | pub fn foo() {}
  | ^^^^^^^^^^^^^^

error: aborting due to 2 previous errors
```

Configuring lints using attributes

```
$ cat lib.rs
#![warn(missing_docs)] // inner attribute

pub fn foo() {}

$ rustc lib.rs --crate-type=lib
warning: missing documentation for crate
--> lib.rs:1:1
|
1 | / #![warn(missing_docs)]
2 | |
3 | | pub fn foo() {}
  | | _____^
  |
note: lint level defined here
--> lib.rs:1:9
|
1 | #![warn(missing_docs)]
  |          ^^^^^^^^^^^^^
  |

warning: missing documentation for a function
--> lib.rs:3:1
|
3 | pub fn foo() {}
```

Multiple attributes and *reason* parameter

```
#![warn(missing_docs)]
#![deny(unused_variables)]

pub fn foo() {}
```

- The *reason* parameter is displayed when the lint is emitted

```
use std::path::PathBuf;
pub fn get_path() -> PathBuf {
    #[allow(unused_mut, reason = "this is only modified on some platforms")]
    let mut file_name = PathBuf::from("git");
    #[cfg(target_os = "windows")] // conditional compilation attribute
    file_name.set_extension("exe");
    file_name
}
```

Capping lints

rustc --cap-lints LEVEL

sets the “lint cap level”: the maximum level for all lints.

```
fn main() {  
    100u8 << 10;    // it should emit `[deny(exceeding_bitshifts)]`  
}
```

```
$ rustc lib.rs --cap-lints warn
```

```
warning: bitshift exceeds the type's number of bits
```

```
--> lib.rs:2:5
```

```
|  
2 |     100u8 << 10;  
|     ^^^^^^^^^^^
```

```
= note: `[warn(exceeding_bitshifts)]` on by default
```

```
warning: this expression will panic at run-time
```

```
--> lib.rs:2:5
```

```
|  
2 |     100u8 << 10;  
|     ^^^^^^^^^^^ attempt to shift left with overflow
```

- Feature used by Cargo when compiling dependencies; it passes `--cap-lints allow`, so if they have any warnings, they do not pollute the output of the build.

Lint Groups

`rustc` has the concept of a **lint group**, where you can toggle several warnings through one name.

For example, the `nonstandard-style` lint sets `non-camel-case-types`, `non-snake-case`, and `non-upper-case-globals` all at once. So these are equivalent:

```
$ rustc -D nonstandard-style
$ rustc -D non-camel-case-types -D non-snake-case -D non-upper-case-globals
```

Main groups:

- **warnings**: All lints that are set to issue warnings
- **deprecated-safe**: Lints for functions which were erroneously marked as safe in the past
- **future-incompatible**: Lints that detect code that has future-compatibility problems
- **keyword-idents**: Lints that detect identifiers which will become keywords in later editions
- **nonstandard-style**: Violation of standard naming conventions
- **refining-impl-trait**: Detects refinement of `impl Trait` return types by trait implementations
- **unused**: Lints that detect things being declared but not used, or excess syntax

When do lints run?

Lints run in different compilation phases, depending on their meaning. Several lints are grouped into passes, which are executed with a single visitor. Others are placed where needed in the code.

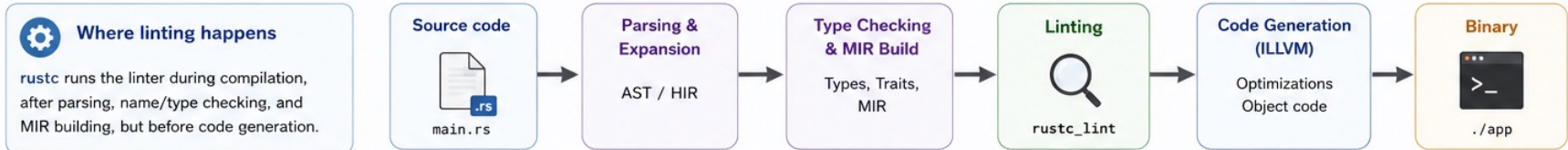
Type	Moment	Available information	Typical use
Pre-expansion	Before macro expansion	rough AST, limited context	edition compatibility and cases sensible to macros, like <code>keyword_ids</code>
Early lint	After macro expansion, before lowering	AST resolved, but types not completed	purely syntactical lints, like <code>unused_parens</code>
Late lint	Towards end of analysis on HIR	HIR, types and richer semantics	Idiomatic or semantic checks, like <code>non_snake_case</code> or <code>invalid_value</code> (uninitialized value, needs types)
MIR / inline	Inside MIR, borrowck or specific code paths	specialized state of the subsystem	<code>arithmetic_overflow</code> , <code>unused_mut</code> , future-compat complex lints
Driver / tool lint	External registration, execution in the same phases as above	depends on the registered pass	Clippy and custom tools via <code>register_lints</code> e <code>rustc_driver</code>

Clippy

- **Clippy** is the official Rust **linter collection**: a large set of additional static analyses (“lints”) built on top of **rustc** that helps developers write more idiomatic, correct, efficient, and maintainable Rust code.
- Conceptually:
 - **rustc** checks whether your program is **valid Rust**.
 - **Clippy** checks whether your Rust is **good Rust**.
- Clippy integrates directly into the compiler infrastructure rather than operating as a separate parser or analyzer.
- Clippy uses the same linting architecture as rustc itself: it registers custom lint passes through the compiler driver and executes them inside the normal lint pipeline.

Linting in Rust

Early feedback for potential issues, style, and correctness—before your code runs.



Lints do not stop compilation by default (except "deny" level lints)

What is a lint?

A lint is a static analysis check performed by the compiler. It can detect:

- likely bugs
- unnecessary code
- performance pitfalls
- style / idioms
- deprecated or future-incompatible code

Lints never change the meaning of your program. They provide suggestions and warnings (or errors, if you opt in).

Example

Code (main.rs)

```
1 fn main() {
2   let unused = 42;           // never used
3
4   let mut x = 0;             // x is never mutated
5   println!("x = {}", x);
6
7   fn takes_string(s: String) {} // takes ownership
8   takes_string("hello".to_string()); // could borrow
9 }
```



\$ cargo check

```
warning: unused variable: `unused`
--> src/main.rs:2:9
2 |   let unused = 42;
  |   ^^^^^^ help: prefix with an underscore: `_unused`
= note: `[warn(unused_variables)]` on by default

warning: variable does not need to be mutable
--> src/main.rs:4:9
4 |   let mut x = 0;
  |   --- help: remove this `mut`
= note: `[warn(unused_mut)]` on by default

note: consider borrowing here
--> src/main.rs:8:18
8 |   takes_string("hello".to_string());
  |                   ^^^^^^^^^^^^^^^^^ help: try: `&"hello".to_string()`
= note: `[warn(clippy::needless_pass_by_value)]` on by default

Finished dev [unoptimized + debuginfo] target(s) in 0.08s
```

Lint levels

Every lint has a level:

- allow** Ignore the lint
- warn** Show a warning (default for most)
- deny** Make it a hard error
- forbid** Forbid it, even if allowed elsewhere

Example:

```
#![warn(unused_variables)]
#![deny(unsafe_code)]

fn main() {
    let _ok = 1; // silenced by underscore
}
```

Common lint groups

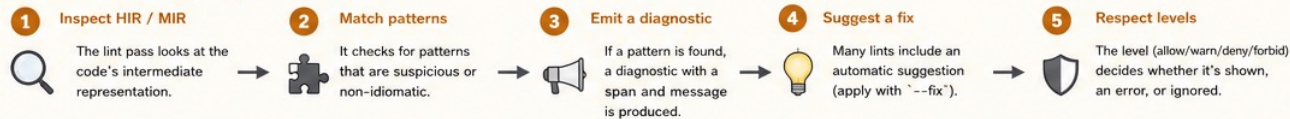
rustc:: Core compiler lints (always available)
clippy:: Additional lints from Clippy
rustdoc:: Lints for documentation
future_incompatible: Future edition breaks
warnings: All lints that are warn by default

Run Clippy: `$ cargo clippy`

Control lints

- At crate level (inner attribute): `#![warn(...)]`
- At module / item level (outer attribute): `#[allow(...)]`
- In code (fine-grained): `#[allow(...)]`
- Command line: `-A/-W/-D/-F <lint>`

How a lint works (simplified)



Why linting matters

- ✓ Find bugs and pitfalls early
- ✓ Improve readability and consistency
- ✓ Learn Rust idioms
- ✓ Avoid future breakage
- ✓ Make code reviews easier

Key idea: Rust's linter gives you fast, actionable feedback at compile time. Treat warnings seriously—your future self (and teammates) will thank you!

Rust Type System

- Nominal
- Substructural (**affine**)
- Algebraic Data Types (struct, enum), with pattern matching
- Parametric generics over types, consts and lifetimes

```
struct ArrayRef<'a, T, const N: usize> {  
    data: &'a [T; N],  
}
```

- Traits with associated functions, types, and constants
- **impl Trait** for anonymous parameters and abstract return types
- Highly restricted coercions

Type Checking (1)

Action	Purpose
Expression checking	Ensure operations are type-correct (assignment compatibility, operator operand types, return types, branch consistency)
Method resolution	Find callable methods (searches inherent impls, trait impls, autoderef/autoref chains, validates receiver type)
Trait obligation checking <code>fn f<T: Clone>(x: T) { x.clone(); }</code>	Validate trait obligations (<code>T: Clone</code> , all implied obligations, where-clauses, supertraits)
Generic argument correctness <code>Array::<i32, 4></code>	Check type/const/lifetime args (arity, constraints, const kinds, well-formedness)

Type Checking (2)

Action	Purpose
Coercion checking	Validate implicit conversions (<code>&mut T → &T</code> , array-to-slice, deref coercions, unsizing coercions)
Pattern checking <code>match x { Some(v) => ... None => ... }</code>	Verify match/destructuring (constructor validity, binding types, exhaustiveness support, enum variant consistency)
Signature checking	Validate fn/closure interfaces (parameter types, return types, ABI consistency, closure call traits (<code>Fn</code> , <code>FnMut</code> , <code>FnOnce</code>))
Operator resolution <code>a + b</code> becomes: <code>Add::add(a, b)</code>	Trait-based operator typing (trait existence, operand compatibility, result type)

Type Checking (3)

Action	Purpose
Autoderef/autoref <code>x.len()</code> May become <code>(*(*x)).len()</code>	Insert implicit refs/derefs (deref chain legality, borrow insertion, mutability correctness)
Well-formedness checking <pre>struct S<T: Copy> { x: T }</pre>	Ensure legal types (bounds, recursive legality, variance rules, sizedness, object safety preconditions)
ADT construction checking <code>Some(3)</code> or <code>Point{x:1,y:2}</code>	Validate enums/structs (field existence, visibility, field types, arity)

Type inference

- Rust type inference is a multi-domain **constraint generation + constraint solving** system over:
 - types,
 - lifetimes (regions),
 - trait obligations,
 - projections,
 - const generics.
- The type inference is based on the standard **Hindley-Milner (HM)** type inference algorithm, extended in various ways to accommodate subtyping, region inference, and higher-ranked types.

Type inference actions (1)

Inference Action	Purpose	Example Constraint
Local variable inference	Infer variable types	<pre>let x = 3;</pre> • $x : ?T1$ // fresh vars • $?T1 = \text{IntVar}$ // by 3 • $\text{IntVar} = i32$ // fallback $\text{IntVar} \rightarrow i32$ $\text{FloatVar} \rightarrow f64$
Return type inference	Infer function result type	<pre>fn f() { 3 }</pre> • $\text{return_type} = \text{body_type}$
Generic argument inference	Infer generic parameters	<pre>let v = Vec::new(); v.push(3);</pre> • $v : \text{Vec}<?T1>$ • $?T1 = i32$ // by method call • $v : \text{Vec}<i32>$
Closure parameter and result inference	Infer closure params/results	<pre>let f = x x + 1;</pre> • $x : ?T1$ • $\text{result} : ?T2$ • $?T1 : \text{Add}<i32>$ // overloading • $\text{Add}::\text{Output}(?T1, i32) = ?T2$ • $?T1 = i32$ // typical solution • $?T2 = i32$

Type inference actions (2)

Inference Action	Purpose	Example Constraint
Method receiver inference	Infer receiver structure	<code>x.push(3)</code> • <code>x : ?T1 // fresh</code> • <code>?T1 = Vec<?T2> // method lookup</code> • <code>?T2 = i32 // argument</code> • <code>?T1 = Vec<i32></code>
Reference inference	Infer borrow type	<code>let r = &x;</code> • Suppose <code>x : i32</code> • <code>r : &'?R i32 // by inference</code> • <code>'?R // fresh region variable</code> • <code>region(x) $\supseteq$ '?R // constraint</code> • <code>x must outlive the borrow</code>
Lifetime inference	Infer region extent	<code>let r = &x;</code> <code>println!("{}", r);</code> • <code>borrow_point $\in$ '?R</code> • <code>use_point $\in$ '?R</code>
Reborrow inference	Restrict nested borrows	<code>let y = &*x;</code> • Suppose <code>x : &'a mut i32</code> • <code>y : &'?R i32 // by inference</code> • <code>'?R $\subseteq$ 'a</code> The reborrow cannot outlive the original borrow

Type inference actions (3)

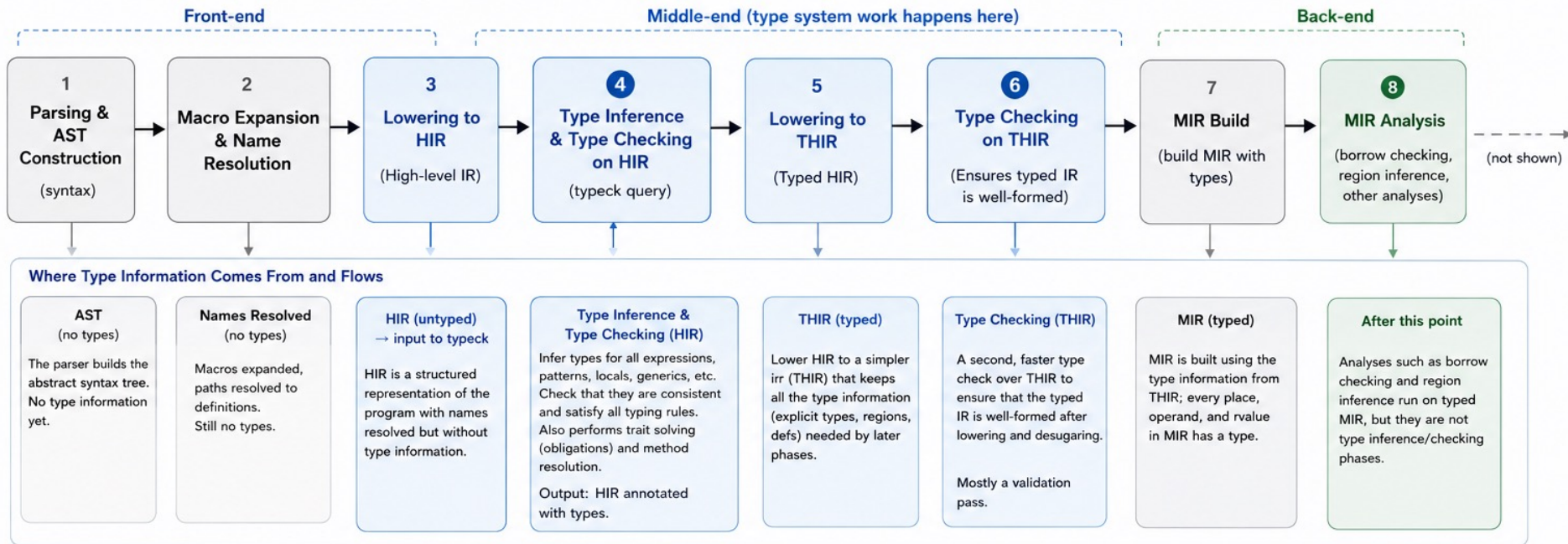
Inference Action	Purpose	Example Constraint
Trait obligation inference	Infer required traits	<code>x.clone()</code> • <code>x : ?T1 // by inference</code> • <code>?T : Clone // constraint</code>
Associated type inference	Resolve projections	<code>Iterator::Item</code> • <code>Suppose ?T1 : Iterator</code> • <code><?T1 as Iterator>::Item = ?T2</code>
Branch unification «Match-arm» type inference	Unify branch types	<code>let y = if cond {3} else {4};</code> • <code>arm1_type = ?T</code> • <code>arm2_type = ?T</code>
Array inference	Infer element types/lengths	<code>let a = [1,2,3];</code> • <code>a : [i32; 3]</code> • all elements same type, • length fixed.
Higher-ranked lifetime (HRTB) inference	Infer higher-ranked lifetimes	<code>for<'a> fn(&'a i32)</code> • bound region variables, • universes, • placeholder regions <code>∀'a . valid(fn(&'a i32))</code>
Subtyping/outlives inference	Infer lifetime containment	<code>let x: &'static i32 = y;</code> • <code>y : &'?R i32</code> • <code>'?R : 'static</code>

Type constraints generated

Constraint kind	Meaning
Equality	$?T = i32$
Subtyping	$'a : 'b$
Trait obligations	$?T : Clone$
Projection equality	$\langle T \text{ as Trait} \rangle :: Assoc = U$
Region containment	$'?R \subseteq 'a$
Const equality	$?N = 4$

Where rustc Performs Type Checking and Type Inference

(ignoring borrow checking)



Key Type-Related Activities in rustc

4 Type Inference & Type Checking on HIR (typeck query)

- ✓ Type inference for expressions, let-bindings, fn bodies
- ✓ Type checking of function items, impls, statics, consts
- ✓ Pattern checking and match exhaustiveness (uses types)
- ✓ Trait solving and obligation checking
- ✓ Method resolution (inherent + trait methods)
- ✓ Generics checking (bounds, where-clauses)
- ✓ Coercions and adjustments
- ✓ Region (lifetime) inference (for type system, not borrow checking)

Produces: HIR with full type annotations

6 Type Checking on THIR

- ✓ Validate that all THIR nodes are well-typed
- ✓ Check invariants after lowering/desugaring
- ✓ Ensure types of expressions, patterns, and statements respect the rules
- ✓ Much cheaper than full inference; acts as a sanity check

Produces: Validated typed THIR

Legend

- Syntax / name resolution (no types yet)
- Type inference / type checking activities
- Typed IR produced / consumed
- IR construction (types already known)
- Later analyses (not type checking/inference)
- Flow of information



Summary

rustc performs type inference and type checking mainly in the front-end / early middle-end:

(1) full inference & checking on HIR (typeck), and (2) a validation type check on THIR.

After MIR is built, subsequent phases use the already computed types but do not perform type inference (borrow checking is not shown).

Notes

- Types are available before MIR is built.
- MIR, and all later passes, rely on the types computed earlier.

Why MIR type checking is run after move analysis?

- Rust performs the MIR type check **after** move analysis because the **MIR type checker needs information produced by move analysis**.
- It's main purpose inside borrow checking is to generate **precise region/lifetime constraints**, and those constraints depend on knowing:
 - which places are still initialized,
 - which moves occurred,
 - which paths are legal to move from,
 - and which locals/places are actually relevant to borrow analysis.
- The ordering is therefore driven by the dependencies between analyses.

About type inference algorithms

- Hindley-Milner type system

https://en.wikipedia.org/wiki/Hindley%E2%80%93Milner_type_system

- Algorithm W: most used, but not the only one:
 - good for pure languages, side effects limited to new variables
 - Algorithm J: uses more side effects, more efficient
 - Lazy unification: collect all constraints visiting the AST, then apply unification

First-Order Unification

Given a finite set $G = \{s_1 \doteq t_1, \dots, s_n \doteq t_n\}$ of potential equations, the algorithm applies rules to transform it to a **substitution**, i.e. an equivalent set of equations of the form $\{x_1 \doteq u_1, \dots, x_m \doteq u_m\}$ where $x_1, \dots, x_m$ are distinct variables and $u_1, \dots, u_m$ are terms containing none of the x_i .

If there is no solution the algorithm terminates with $\perp$.

$G \{x \mapsto t\}$ denotes the operation of substituting all occurrences of variable x in problem G with term t .

Constant symbols are regarded as function symbols having zero arguments.

Martelli–Montanari algorithm (1976-82)

The algorithm computes the **most general unifier**, i.e. a substitution S such that $S(s_i) = S(t_i)$ for all $s_i = t_i$ in G . If there is no solution the algorithm terminates with $\perp$.

$G \cup \{t \doteq t\} \Rightarrow G$	<i>delete</i>
$G \cup \{f(s_0, \dots, s_k) \doteq f(t_0, \dots, t_k)\} \Rightarrow G \cup \{s_0 \doteq t_0, \dots, s_k \doteq t_k\}$	<i>decompose</i>
$G \cup \{f(s_0, \dots, s_k) \doteq g(t_0, \dots, t_m)\} \Rightarrow \perp$	if $f \neq g$ or $k \neq m$ <i>conflict</i>
$G \cup \{f(s_0, \dots, s_k) \doteq x\} \Rightarrow G \cup \{x \doteq f(s_0, \dots, s_k)\}$	<i>swap</i>
$G \cup \{x \doteq t\} \Rightarrow G\{x \mapsto t\} \cup \{x \doteq t\}$	if $x \notin \text{vars}(t)$ and $x \in \text{vars}(G)$ <i>eliminate</i> ^[note]
$G \cup \{x \doteq f(s_0, \dots, s_k)\} \Rightarrow \perp$	if $x \in \text{vars}(f(s_0, \dots, s_k))$ <i>check</i>

Other topics around Rust

- Semantics of the language: **RustBelt**

<https://plv.mpi-sws.org/rustbelt/pop18/>

- **Polonious**: a new borrow-checker

<https://smallcultfollowing.com/babysteps/blog/2018/04/27/an-alias-based-formulation-of-the-borrow-checker/>

- Unique bugs in Rust / rustc

<https://dl.acm.org/doi/10.1145/3763800>

RustBelt: Securing the Foundations of the Rust Programming Language

Rust is a new systems programming language that promises to overcome the seemingly fundamental tradeoff between high-level safety guarantees and low-level control over resource management. Unfortunately, none of Rust's safety claims have been formally proven, and there is good reason to question whether they actually hold. Specifically, Rust employs a strong, [ownership-based type system](#), but then extends the expressive power of this core type system through libraries [that internally use unsafe features](#).

In this paper, we give the first formal (and machine-checked) safety proof for a language representing a realistic subset of Rust. Our proof is extensible in the sense that, for each new Rust library that uses unsafe features, we can say what verification condition it must satisfy in order for it to be deemed a safe extension to the language.

We have carried out this verification for some of the most important libraries that are used throughout the Rust ecosystem.

Polonius: An alias-based formulation of the borrow checker

Conservative extension of NLL Borrow-Checker

- Regions are sets of loans
- The biggest shift in this new approach is that when you have a type like `&'a i32`, the meaning of `'a` changes:
 - In the system described in the NLL RFC, `'a` – called a lifetime – ultimately corresponded to some portion of the source program or control-flow graph.
 - Under *this* proposal, `'a` – which I will be calling a region – instead corresponds to a set of **loans** – that is, a set of borrow expressions, like `&x` or `&mut v`. The idea is that if a reference `r` has type `&'a i32` then invalidating the **terms of any of the loans** in `'a` would invalidate `r`.

An Empirical Study of Rust-Specific Bugs in the rustc Compiler

[J. ACM, Vol. 37, No. 4. August 2025](#)

Study of bugs of **rustc** due to unique features of Rust, including trait solving, borrow checking, and specific optimizations.

Based on issues and fixes reported between 2022 and 2024, with a manual review of 301 valid issues.

Findings:

1. **rustc** bugs primarily arise from Rust's type system and lifetime model, with frequent errors in the HIR and MIR modules due to complex checkers and optimizations;
2. bug-revealing test cases often involve unstable features, advanced trait usages, lifetime annotations, standard APIs, and specific optimization levels;
3. while both valid and invalid programs can trigger bugs, existing testing tools struggle to detect non-crash errors, underscoring the need for further advancements in **rustc** testing.

uHaskell

- Subset of Haskell to explain type inference.
 - Haskell and ML both have **overloading**
 - Will do not consider overloading now

```
<decl> ::= <name> <pat> = <exp>
<pat>  ::= Id | (<pat>, <pat>) | <pat> : <pat> | []
<exp>  ::= Int | Bool | [] | Id | (<exp>)
          | <exp> <op> <exp>
          | <exp> <exp> | (<exp>, <exp>)
          | if <exp> then <exp> else <exp>
```

Type Inference: Basic Idea

- Example

```
f x = 2 + x          -- a simple declaration
```

- What is the type of **f**?

+ has type: $\text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$

(with overloading would be $\text{Num } a \Rightarrow a \rightarrow a \rightarrow a$)

2 has type: Int

Since we are applying **+** to **x** we need $x :: \text{Int}$

Therefore **f x = 2 + x** has type $\text{Int} \rightarrow \text{Int}$

```
f x = 2 + x          -- a simple declaration
> f :: Int -> Int
```

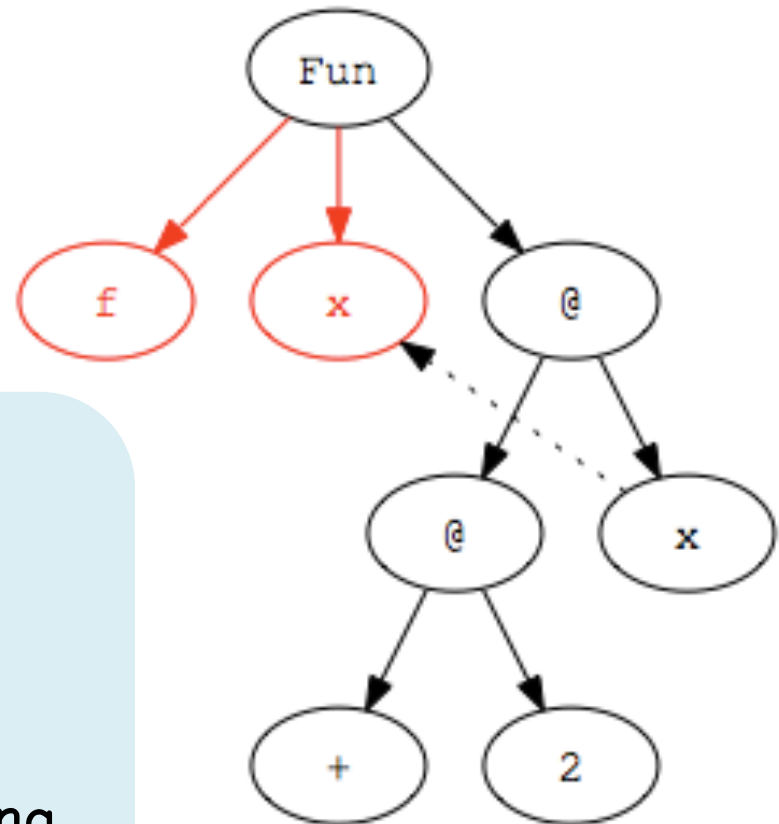
Type Inference Algorithm

- Parse program to build parse tree
- Assign type variables to nodes in tree
- Generate constraints:
 - From environment: constants (**2**), built-in operators (**+**), known functions (**tail**).
 - From shape of parse tree: e.g., application and abstraction nodes.
- Solve constraints using *unification*
- Determine types of top-level declarations

Step 1: Parse Program

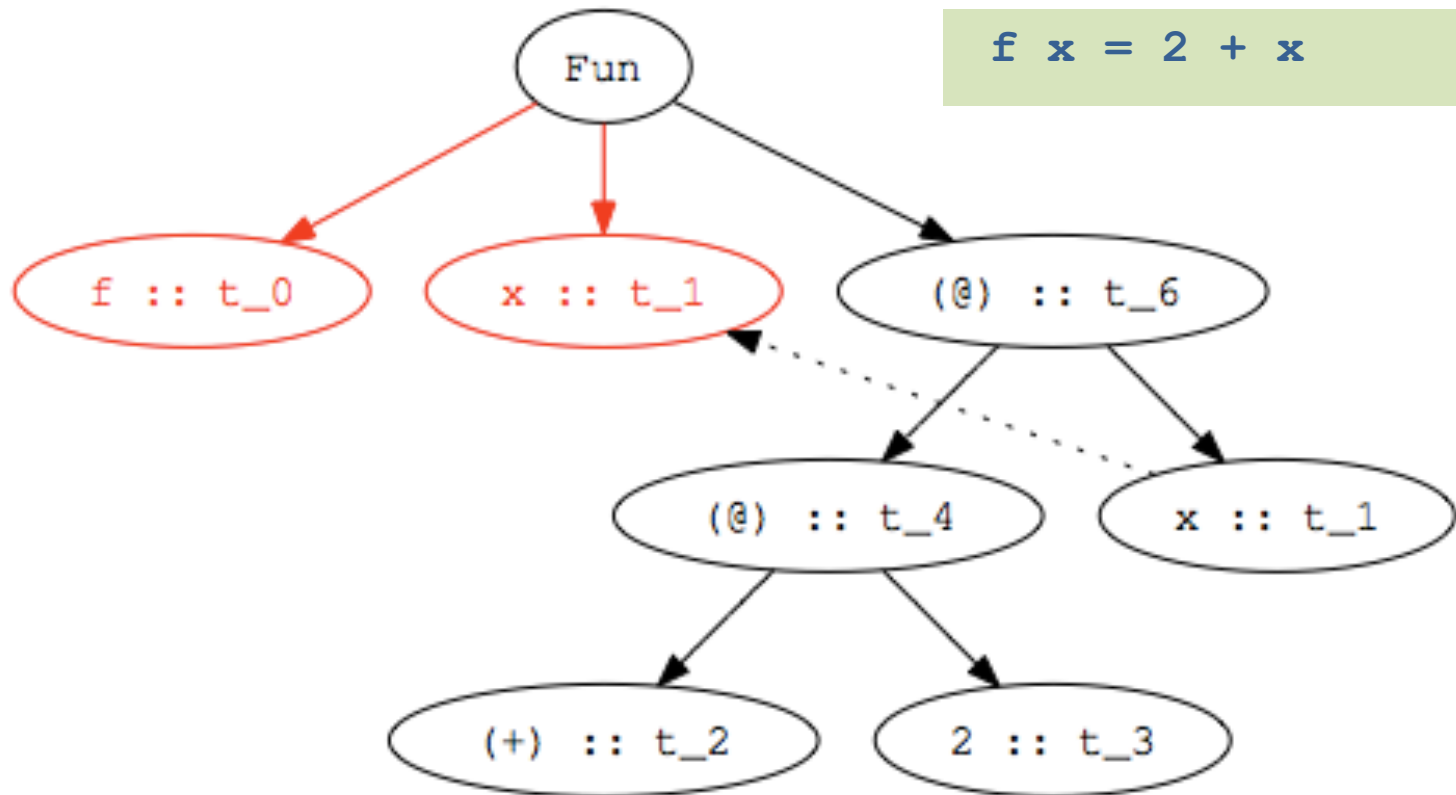
- Parse program text to construct parse tree.

`f x = 2 + x`



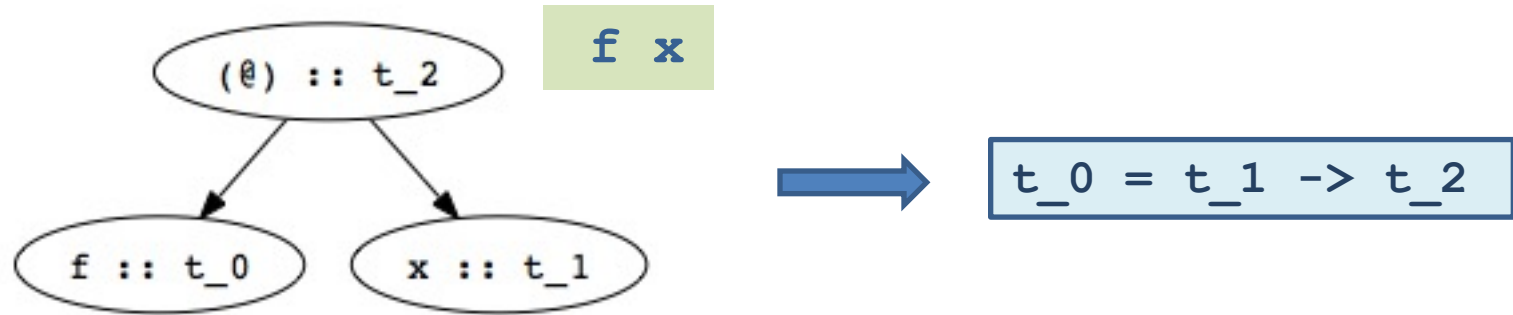
- Binary @-nodes to represent application
- Ternary **Fun**-node for function definitions
- Infix operators are converted to Curried function application during parsing: `2 + x` $\rightarrow$ `(+) 2 x`

Step 2: Assign type variables to nodes



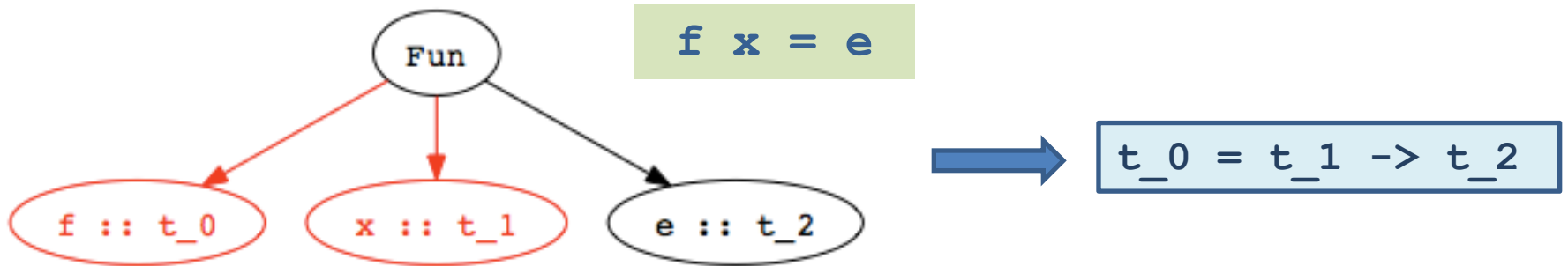
Variables are given same type as binding occurrence.

Constraints from Application Nodes



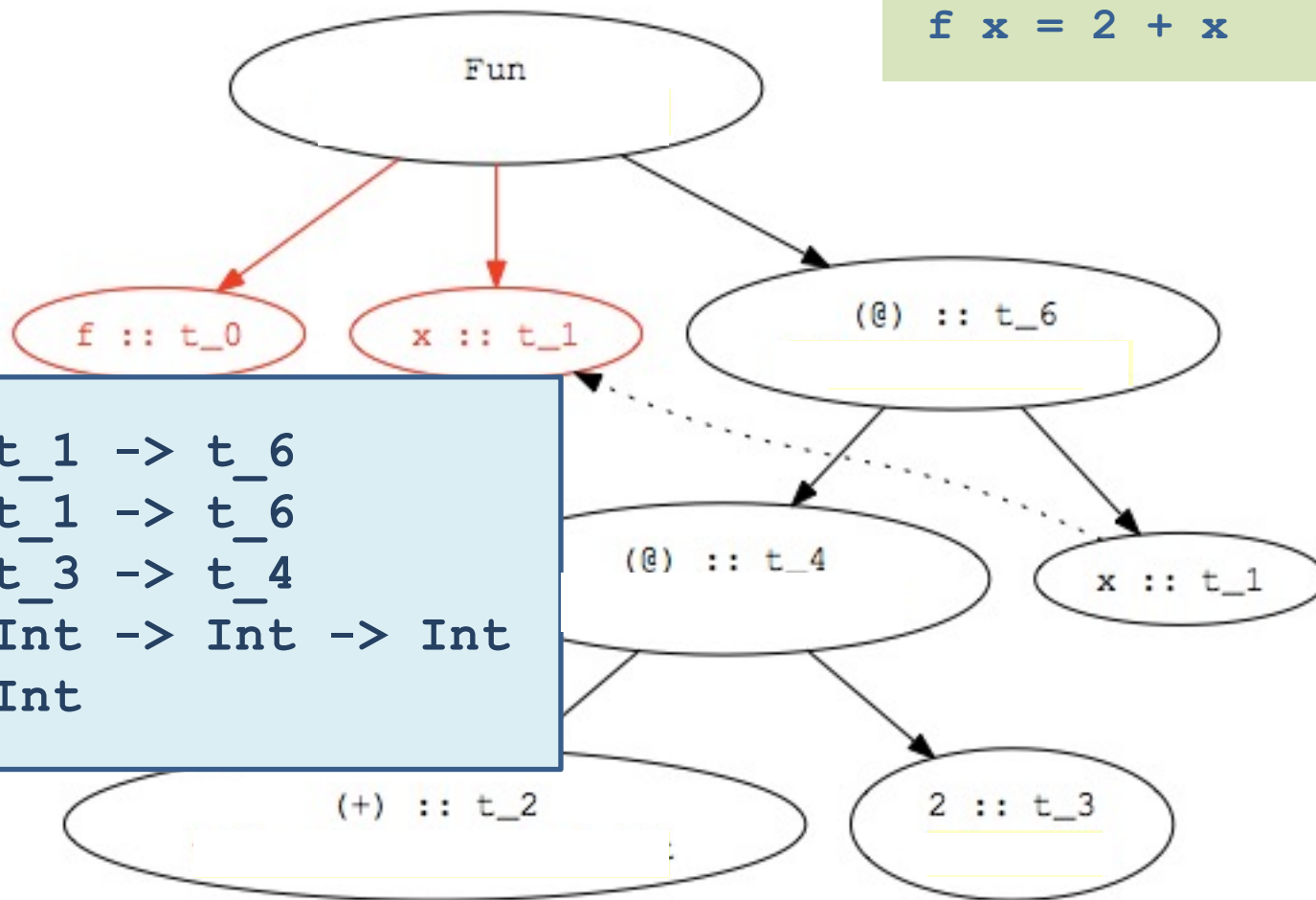
- Function application (apply f to x)
 - Type of f (t_0 in figure) must be **domain** $\rightarrow$ **range**.
 - **Domain** of f must be type of argument x (t_1)
 - **Range** of f must be result of application (t_2)
 - **Constraint**: $t_0 = t_1 \rightarrow t_2$

Constraints from Abstractions



- Function declaration:
 - Type of f (t_0) must **domain** $\rightarrow$ **range**
 - **Domain** is type of abstracted variable x (t_1)
 - **Range** is type of function body e (t_2)
 - **Constraint**: $t_0 = t_1 \rightarrow t_2$

Step 3: Add Constraints



Step 4: Solve Constraints

```
t_0 = t_1 -> t_6  
t_4 = t_1 -> t_6  
t_2 = t_3 -> t_4  
t_2 = Int -> Int -> Int  
t_3 = Int
```

$t_3 \rightarrow t_4 = \text{Int} \rightarrow (\text{Int} \rightarrow \text{Int})$

```
t_3 = Int  
t_4 = Int -> Int
```

```
t_0 = t_1 -> t_6  
t_4 = t_1 -> t_6  
t_4 = Int -> Int  
t_2 = Int -> Int -> Int  
t_3 = Int
```

$t_1 \rightarrow t_6 = \text{Int} \rightarrow \text{Int}$

```
t_0 = Int -> Int  
t_1 = Int  
t_6 = Int  
t_4 = Int -> Int  
t_2 = Int -> Int -> Int  
t_3 = Int
```

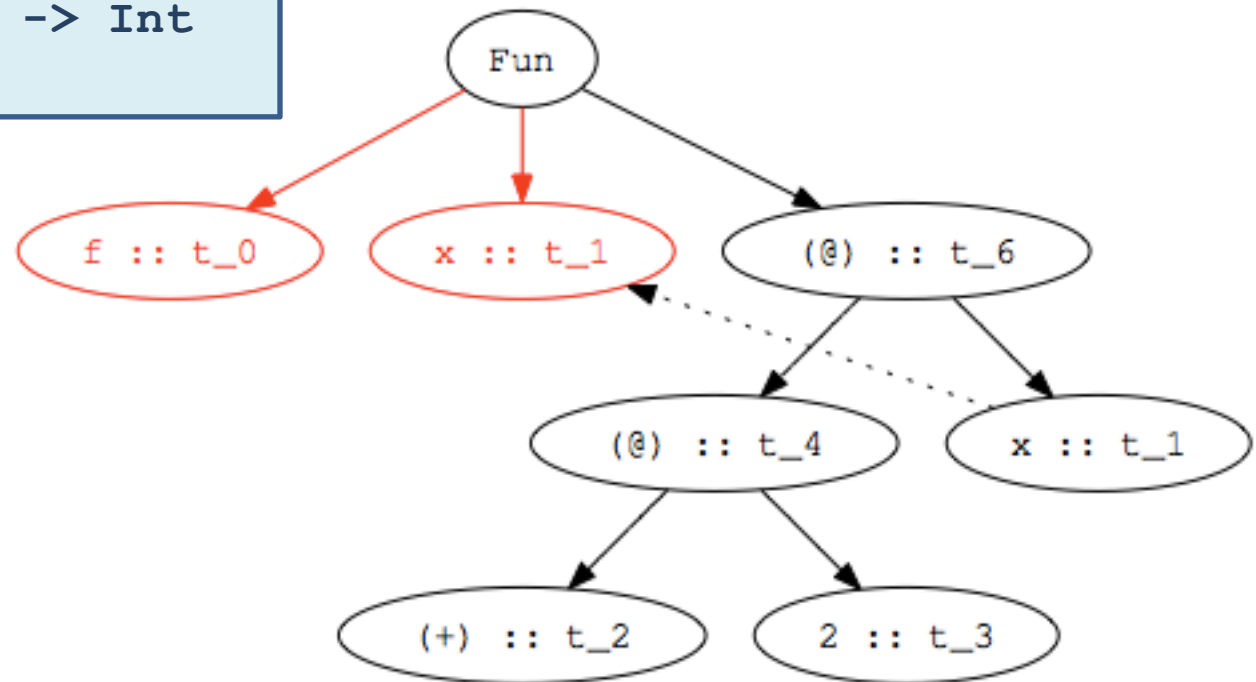
```
t_1 = Int  
t_6 = Int
```

Step 5:

Determine type of declaration

```
t_0 = Int -> Int  
t_1 = Int  
t_6 = Int  
t_4 = Int -> Int  
t_2 = Int -> Int -> Int  
t_3 = Int
```

```
f x = 2 + x  
> f :: Int -> Int
```

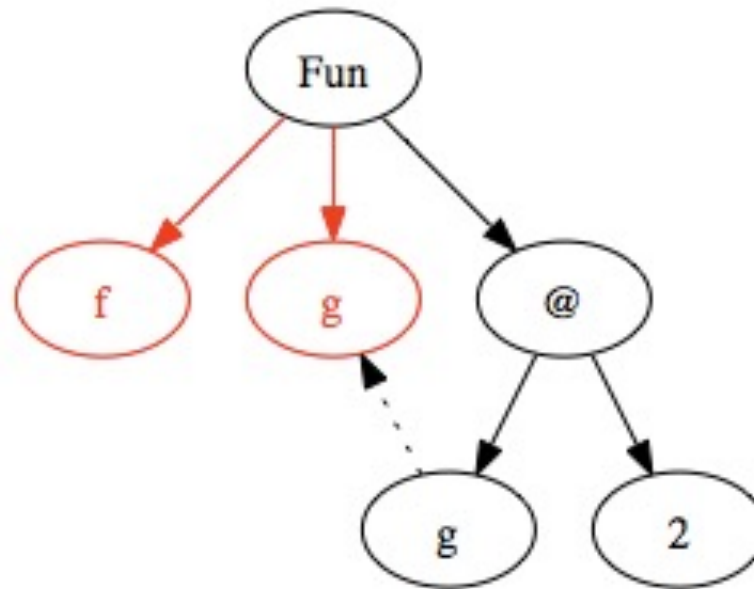


Inferring Polymorphic Types

- Example:

```
f g = g 2  
> f :: (Int -> t_4) -> t_4
```

- Step 1:
Build Parse Tree

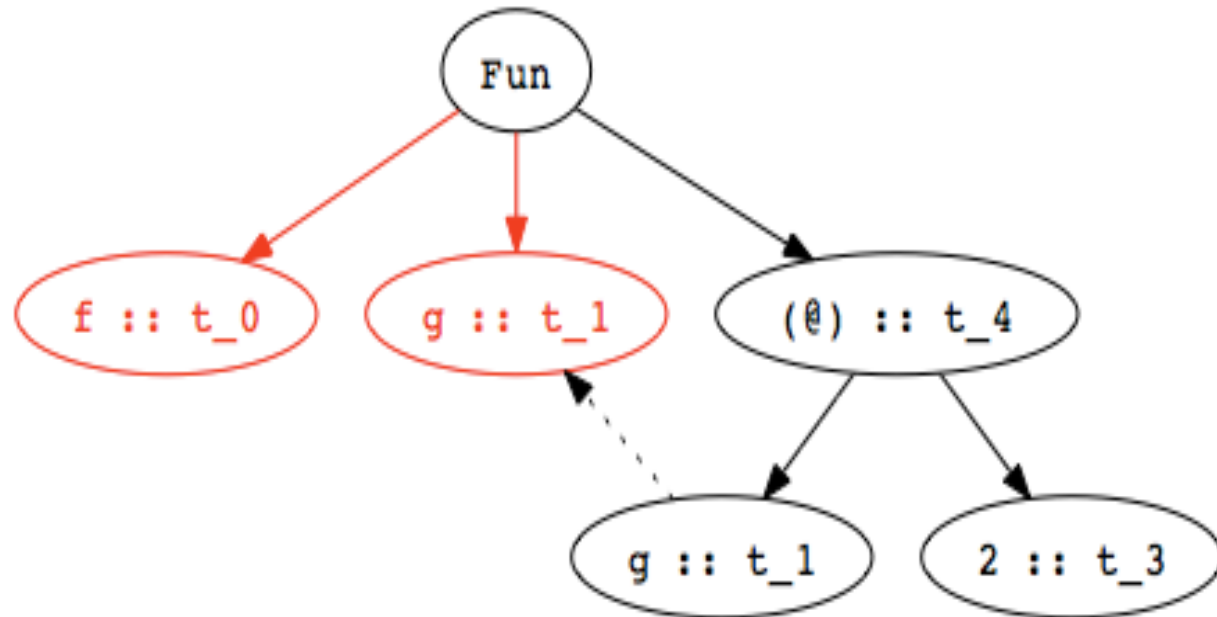


Inferring Polymorphic Types

- Example:

```
f g = g 2  
> f :: (Int -> t_4) -> t_4
```

- Step 2:
Assign type variables



Inferring Polymorphic Types

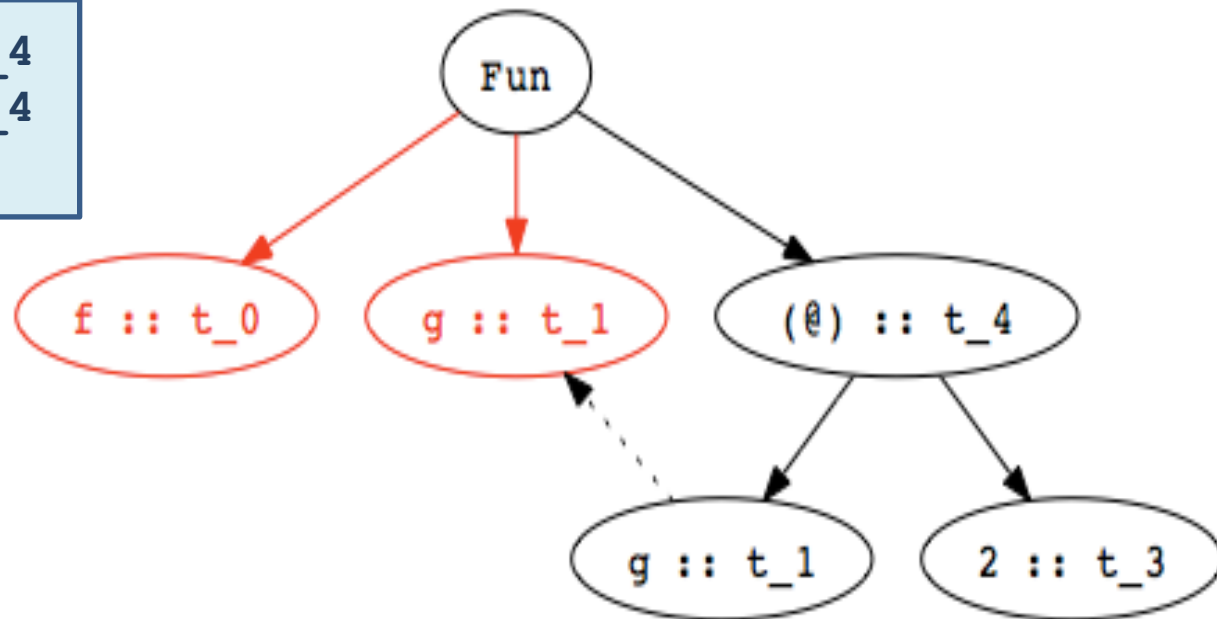
- Example:

```
f g = g 2  
> f :: (Int -> t_4) -> t_4
```

- Step 3:

Generate constraints

```
t_0 = t_1 -> t_4  
t_1 = t_3 -> t_4  
t_3 = Int
```



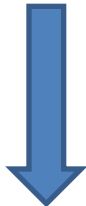
Inferring Polymorphic Types

- Example:

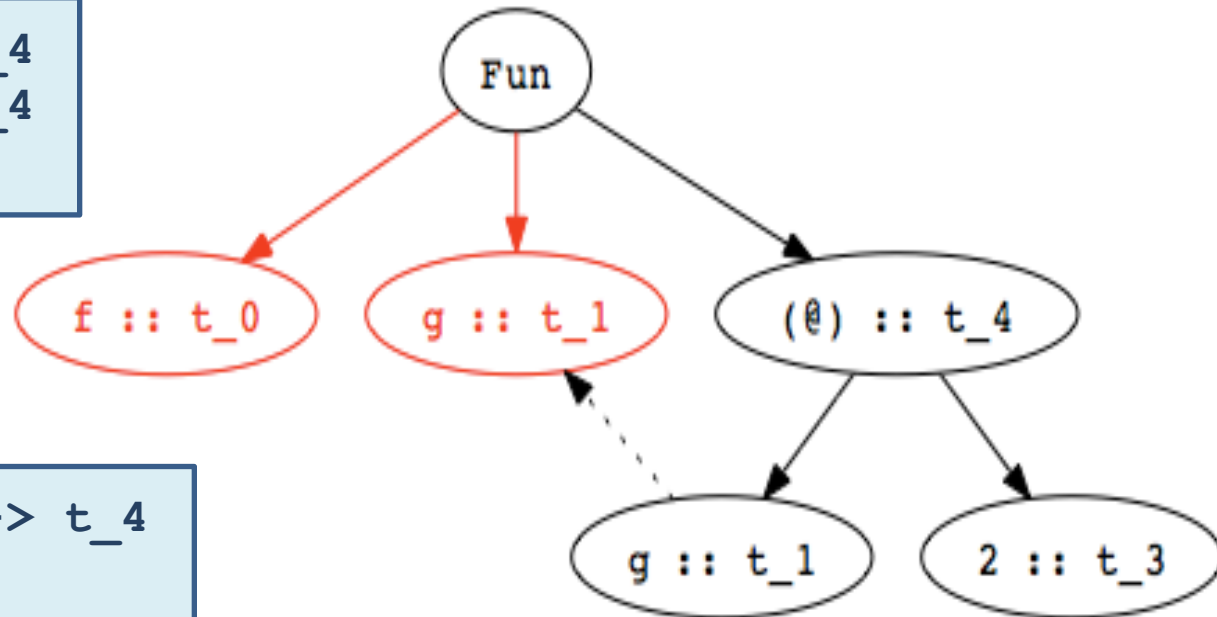
```
f g = g 2  
> f :: (Int -> t_4) -> t_4
```

- Step 4:
Solve constraints

```
t_0 = t_1 -> t_4  
t_1 = t_3 -> t_4  
t_3 = Int
```



```
t_0 = (Int -> t_4) -> t_4  
t_1 = Int -> t_4  
t_3 = Int
```



Inferring Polymorphic Types

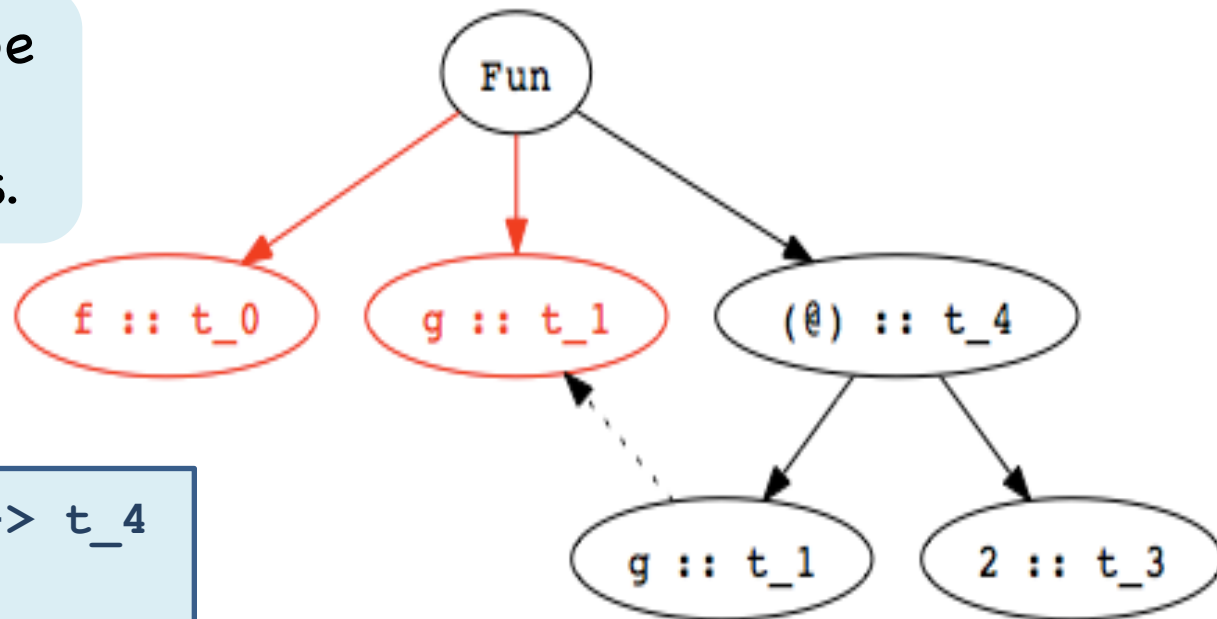
- Example:

```
f g = g 2  
> f :: (Int -> t_4) -> t_4
```

- Step 5:

Determine type of top-level declaration

Unconstrained type variables become polymorphic types.



```
t_0 = (Int -> t_4) -> t_4  
t_1 = Int -> t_4  
t_3 = Int
```

Using Polymorphic Functions

- Function:

```
f g = g 2  
> f :: (Int -> t_4) -> t_4
```

- Possible applications:

```
add x = 2 + x  
> add :: Int -> Int
```

```
f add  
> 4 :: Int
```

```
isEven x = mod (x, 2) == 0  
> isEven :: Int -> Bool
```

```
f isEven  
> True :: Bool
```

Polymorphic Datatypes

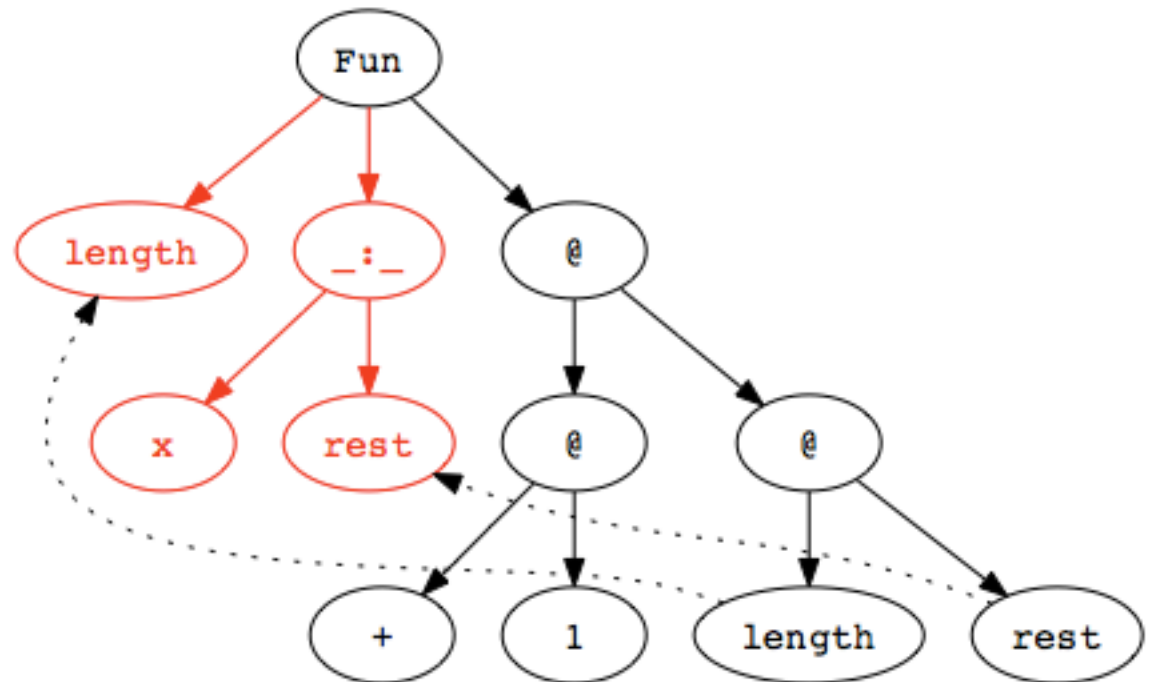
- Functions may have multiple clauses

```
length [] = 0
length (x:rest) = 1 + (length rest)
```

- Type inference
 - Infer separate type for each clause
 - Combine by adding constraint that all clauses must have the same type
 - Recursive calls: function has same type as its definition

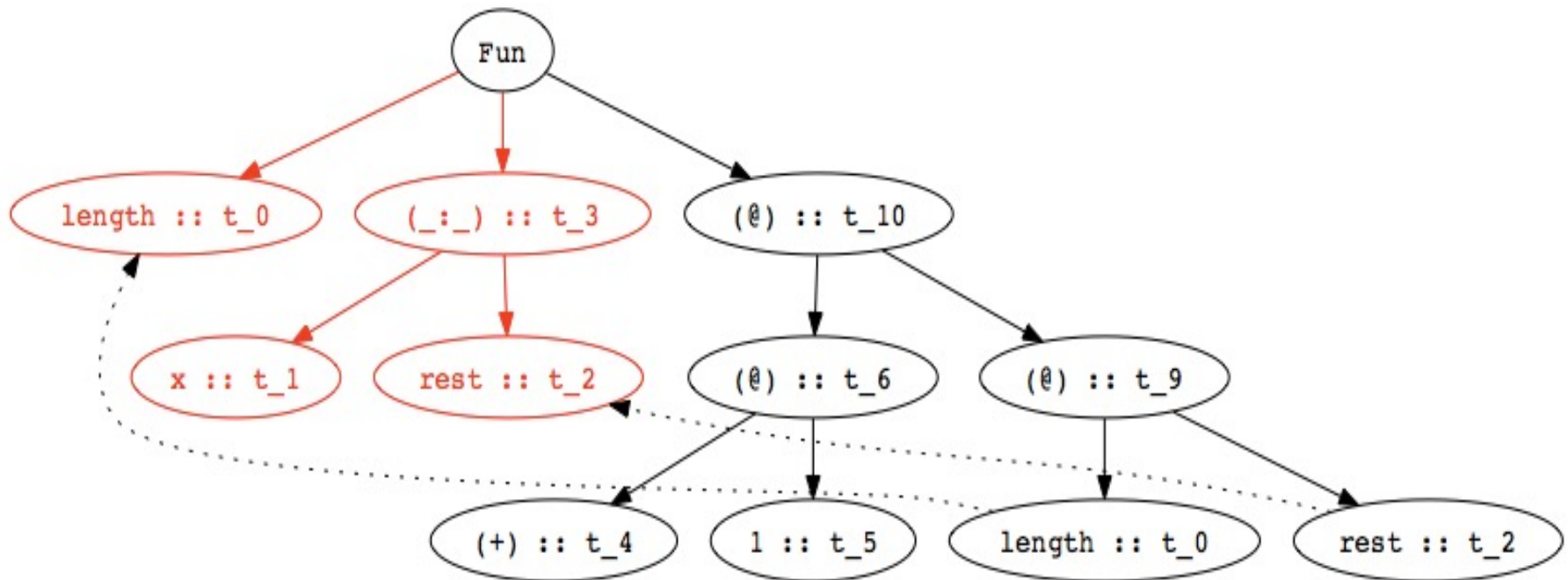
Type Inference with Datatypes

- Example: `length (x:rest) = 1 + (length rest)`
- Step 1: Build Parse Tree



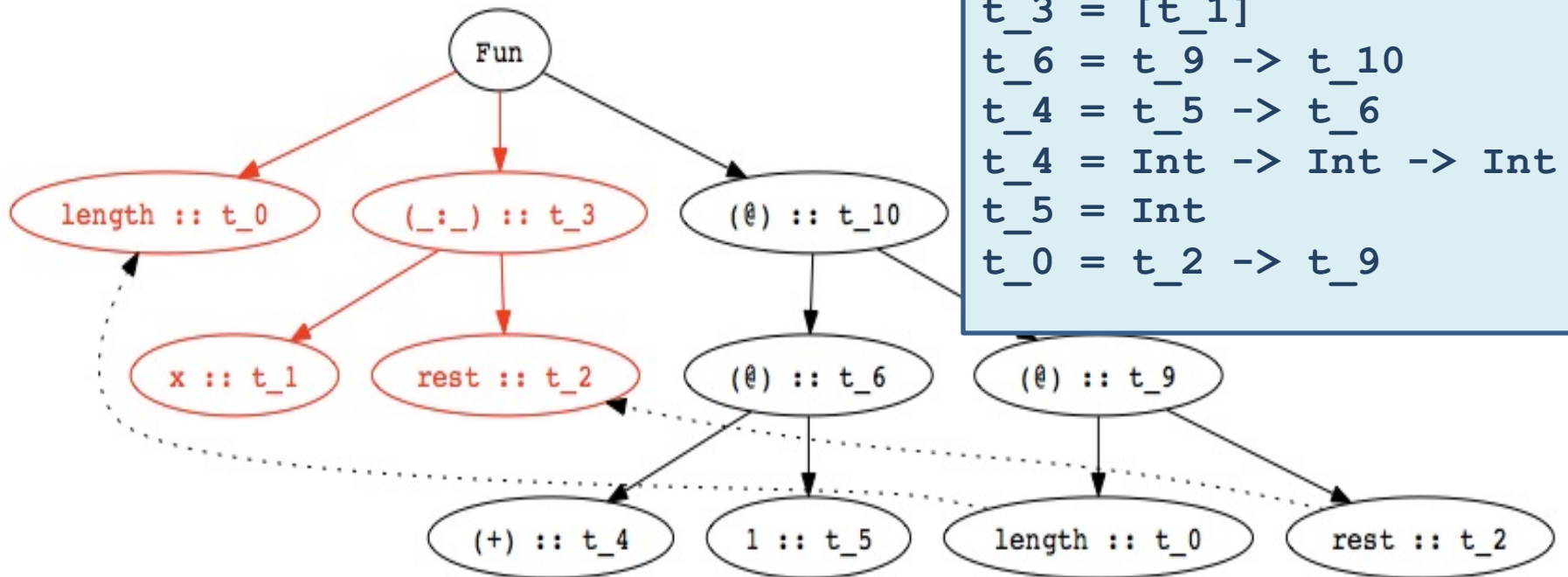
Type Inference with Datatypes

- Example: `length (x:rest) = 1 + (length rest)`
- Step 2: Assign type variables



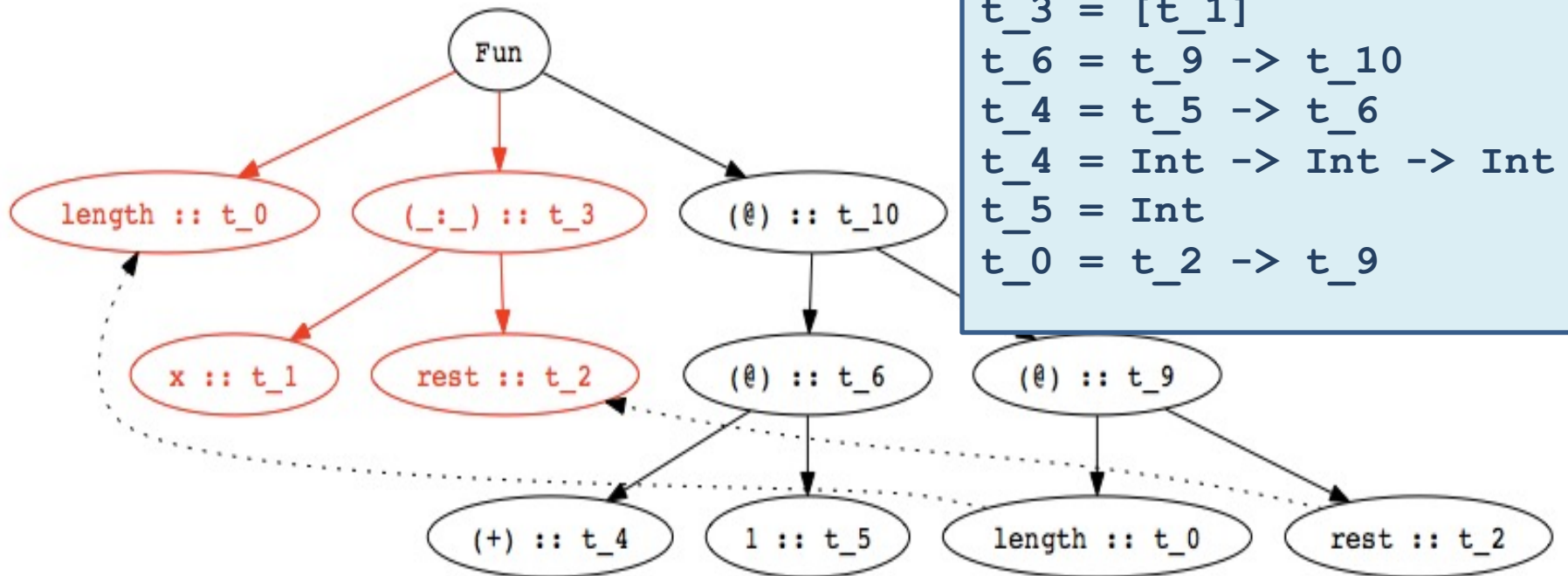
Type Inference with Datatypes

- Example: `length (x:rest) = 1 + (length rest)`
- Step 3: Generate constraints



Type Inference with Datatypes

- Example: `length (x:rest) = 1 + (length rest)`
- Step 3: Solve Constraints



`t_0 = [t_1] -> Int`